

Hak-Bul - Veri Akisi & Qdrant Mimarisi

RAG pipeline veri katmani: ham veri -> chunking -> embedding -> Qdrant Cloud

1. Genel Veri Akisi

Ham veriden sorgu yanitina kadar pipeline 4 asamalidir:

HAM VERI -> CHUNKING -> EMBEDDING -> ARAMA

Mevzuat.gov.tr (XML / scraping)

chunk_kanun.py -> madde bazli bolme -> metadata ekle

```
-> data/processed/4857_is_kanunu_chunks.json
-> data/processed/5510_sgg_kanunu_chunks.json
-> data/processed/6098_turk_borclar_kanunu_chunks.json
```

Bu JSON'lar Qdrant'a GITMIYOR. retriever.py icindeki keyword-based local index'e yukleniyor. Kanun maddeleri madde numarasıyla arandigindan embedding gerekmez, dogrudan token eslemesi yeterlidir.

Yargitay.gov.tr (HTML scraping)

chunk_yargitay.py -> ozet / gerekce / sonuc bolme -> metadata ekle

```
-> data/processed/yargitay_chunks.json
```

Bu JSON Qdrant'a GIDIYOR. 'Kidem tazminati' sorusuna ilgili emsal kararini bulmak icin semantik benzerlik gereklidir; keyword aramasi bu is icin yeterli degil.

2. Neyin Nerede Tutuldugu

Veri	Nerede	Arama Yontemi	Neden
Kanun metinleri (4857, 5510, 6098)	Local JSON data/processed/	Keyword (token eslesmesi)	Madde no ile direkt esleme yeterli
Yargitay kararları (500-1000 karar)	Qdrant Cloud hukuk_chunks	Semantic (cosine similarity)	Serbest metin aramasi icin embedding lazim

3. load_qdrant.py Nasil Calisir?

Script tek bir is yapar: hazır Yargitay JSON'ini alır, her chunk'i multilingual-e5-base ile encode eder, Qdrant Cloud'a yukler. Ham veriye veya scraper'a dogrudan bagimlilik■ yoktur.

Kullanım

```
# Eski model vektorlerini temizleyip yeniden yukle (model degistiginde zorunlu)
python scripts/load_qdrant.py --recreate --input data/processed/

# Yuklenecek chunk sayisini once kontrol et (gercek yukleme yapmaz)
python scripts/load_qdrant.py --dry-run

# Tek dosya yukle
python scripts/load_qdrant.py --recreate --input data/processed/yargitay_chunks.json
```

İc İşleyiş

Adım	İşlem	Detay
1	JSON Yukle	data/processed/yargitay_*.json dosyalarını oku

2	Filtrele	Sadece kaynak_turu="yargitay_karari" chunk'larini al (kanun chunk'lari atlanir - local index kullaniyor)
3	Koleksiyon	--recreate: eski koleksiyonu sil, yeniden olustur (vector_size=768, distance=Cosine - degismez)
4	Encode	model.encode("passage: " + metin) multilingual-e5-base icin zorunlu prefix
5	Batch Upload	64'luk batch'ler halinde Qdrant'a upsert chunk_id'den deterministik point ID uretilir
6	Dogrula	Koleksiyon ozeti: toplam vektor sayisi, boyut, distance

4. Yargitay Chunk JSON Formati

chunk_yargitay.py (Yagiz / Mustafa sorumlulugu) asagidaki formati uretmeli. load_qdrant.py bu alanlari bekliyor:

```
{
  "chunk_id": "yargitay_2023_1234_gerekce", // zorunlu, benzersiz
  "kaynak_turu": "yargitay_karari", // zorunlu
  "metin": "...kidem tazminatina...", // zorunlu - encode edilecek
  "karar_no": "2023/1234",
  "daire": "9. Hukuk Dairesi",
  "karar_bolumu": "gerekce", // ozet | gerekce | sonuc
  "tarih": "2023-05-12",
  "yil": 2023,
  "url": "https://karararama.yargitay.gov.tr/..."
}
```

5. Qdrant'i Ne Zaman Yeniden Yuklemeli?

Senaryo	Yapilacak
Embedding modeli degisti (mpnet -> multilingual-e5-base)	--recreate zorunlu Eski vektorler farkli uzayda, skorlar anlamsiz
Yeni Yargitay kararlari eklendi	--recreate VEYA sadece yeni chunk'lari upsert etmek yeterli
Kanun metinleri guncellendi	Qdrant'a dokunma - local JSON'i guncelle + backend restart
Koleksiyon bozuldu / test ortami sifirlama	--recreate ile sifirla

6. Qdrant Koleksiyon Parametreleri

Parametre	Deger	Aciklama
Koleksiyon adi	hukuk_chunks	settings.COLLECTION_NAME (.env ile degistirilebilir)
Vector size	768	multilingual-e5-base cikti boyutu (mpnet ile ayni - degismedi)
Distance metric	Cosine	Semantic benzerlik icin standart secim
Icerik	Yargitay kararlari	Kanun metinleri local JSON'da, Qdrant'ta degil
Hedef vektor sayisi	~2.000-3.000	500-1000 karar x ozet+gerekce+sonuc chunk

Tahmini boyut	~18 MB	Qdrant free tier (1 GB) için rahatlıkla yeterli
---------------	--------	---